

Topic: At the same age, are younger generations less likely to own homes than previous generations?

Exploratory Data Analysis (EDA) code:

```
In [1]: import pandas as pd
import numpy as np

df = pd.read_csv("regression.csv")
```

```
In [2]: df.tail()
```

Out[2]:

	YEAR	OWNERSHP	OWNERSHPD	PERWT	AGE	MARST	EDUC	EDUCD	EMPSTAT	EMPSTATD	INCTOT	30 year fixed rate	
1070052	2024	1	12	63.0	48	5	6	63	1	10	42000	6.721154	4
1070053	2024	1	13	89.0	64	1	6	65	3	30	27000	6.721154	4
1070054	2024	2	22	65.0	39	4	11	114	1	10	80300	6.721154	4
1070055	2024	1	13	130.0	15	6	2	26	0	0	1200	6.721154	4
1070056	2024	1	12	96.0	66	1	10	101	3	30	53400	6.721154	4



```
In [3]: df = df.rename(columns={'30 year fixed rate':'fixed_rate_30'})
```

```
In [4]: # 0 means missing or not applicable, so I excluded it in my analysis.
df = df[df['OWNERSHP']!=0]

df['Ownership_Status'] = df['OWNERSHP'].map({1:1,2:0})
```

```
In [5]: # For better interpretability
def marst_group(x):
    if x in [1, 2]:
        return 'Married'
    elif x in [3, 4]:
        return 'Separated/Divorced'
    elif x == 5:
        return 'Widowed'
    elif x == 6:
        return 'Never married'
```

```
In [6]: df['Marital_Status'] = df.MARST.apply(lambda x : marst_group(x))
```

```
In [7]: # For better interpretability
def degree(x):
    if x < 101:
        return "No college Degree"
    elif 101 <= x <=113:
        return "Bachelors Degree"
    elif 114 <= x <=116:
        return "Degrees beyond a bachelors degree"
    else:
        return "Missing"
```

```
In [8]: df['Degree'] = df.EDUCD.apply(lambda x : degree(x))
```

```
In [9]: # For better interpretability
def employment(x):
    if x ==0:
        return "N/A"
    elif x ==1:
        return "Employed"
    elif x ==2:
        return "Unemployed"
    elif x == 3:
        return "Not in labor force"
    else:
        return "Unknown"
```

```
In [10]: df['Employment_Status'] = df.EMPSTAT.apply(lambda x : employment(x))
```

```
In [11]: df['birth_year'] = df['YEAR'] - df['AGE']
```

```
In [12]: # To classify their generations
def generation(x):
    if 1946 <= x <= 1964:
        return 'Boomers'
    elif 1965 <= x <= 1980:
        return 'Gen X'
    elif 1981 <= x <= 1996:
        return 'Millennials'
    elif 1997 <= x <= 2012:
        return 'Gen Z'
    elif x > 2012:
        return 'Gen Alpha'
    else:
        return 'Pre-Boomer'
```

```
In [13]: df['Generation_Category'] = df['birth_year'].apply(lambda x : generation(x))
```

```
In [14]: #According to the IPUMS, these numbers are specific variable codes, which stand for N/A.
# To reduce further confusion, I decided to drop them
df = df[~df['INCTOT'].isin([9999999, 999998])]
```

```
In [15]: df.head()
```

Out[15]:

	YEAR	OWNERSHP	OWNERSHPD	PERWT	AGE	MARST	EDUC	EDUCD	EMPSTAT	EMPSTATD	INCTOT	fixed_rate_30	MSI
0	1970	1	10	100.0	45	6	6	60	1	10	9050	NaN	1
2	1970	1	10	100.0	70	5	2	26	3	30	0	NaN	1
3	1970	1	10	100.0	46	1	6	60	3	30	0	NaN	1
6	1970	2	22	100.0	74	5	1	17	3	30	450	NaN	1
7	1970	2	22	100.0	19	6	1	16	3	30	1950	NaN	1



```
In [16]: # Drop original variables to avoid redundancy and multicollinearity after feature engineering
df = df.drop(['OWNERSHP', 'OWNERSHPD', 'MARST', 'EDUC', 'EDUCD', 'EMPSTAT', 'EMPSTATD'], axis=1)
```

```
In [17]: # the dataset has no interest rate and MSPUS for 1970, so I decided to drop it.
df = df[df['YEAR']!=1970]
# only adults
df = df[df['AGE']>=18]
```

```
In [18]: df['age_group'] = pd.cut(df['AGE'], bins=[0, 25, 35, 45, 55, 65, 100], labels=['18-25', '26-35', '36-45', '46-55', '56-100'])
```

Interpretation

This plot shows a clear divergence between mortgage rates and home prices over the past decades. While interest rates generally declined as home prices increased, this relationship is not consistent during periods of economic disruption. For example, during the post pandemic period, both housing prices and interest rates rise simultaneously, indicating a significant and unusual affordability challenge.

```
In [19]: import cpi
#taking inflation into account
years = df['YEAR'].unique()

cpi_factors = {
    year: cpi.inflate(1, year, to=2024)
    for year in years
}

df['cpi_factor'] = df['YEAR'].map(cpi_factors)

df['adjusted_INCTOT'] = df['INCTOT'] * df['cpi_factor']
df['adjusted_MSPUS'] = df['MSPUS'] * df['cpi_factor']
```

```
In [20]: df.groupby('YEAR')[['adjusted_MSPUS', 'adjusted_INCTOT']].mean()
```

Out[20]:

	adjusted_MSPUS	adjusted_INCTOT
YEAR		
1980	246497.120752	38399.715205
1990	293528.421576	45128.174165
2000	305218.304007	53432.099675
2001	306603.985884	54411.306459
2002	324369.072957	55686.248128
2003	327540.756114	53923.250413
2004	362261.807041	54441.487178
2005	379944.357143	54677.507370
2006	379274.274554	54109.036805
2007	370586.376856	56737.939602
2008	334446.384630	55629.249322
2009	315316.392277	55000.636718
2010	320369.722915	53029.806307
2011	313634.612495	50454.487204
2012	333918.097163	52136.543124
2013	358486.132741	53747.087936
2014	378668.533620	53694.027618
2015	389303.802470	56179.820517
2016	398798.185574	57169.723626
2017	412619.026701	57879.239845
2018	406341.477836	57276.388444

	adjusted_MSPUS	adjusted_INCTOT
YEAR		
2019	392944.070571	60865.683449
2020	397730.565355	62090.401783
2021	443380.769089	59175.412415
2022	464067.425979	58433.947620
2023	439105.095224	59939.016675
2024	418975.000000	59604.903889

```
In [21]: df.groupby('Generation_Category')['adjusted_INCTOT'].median()
```

```
Out[21]: Generation_Category
Boomers      40801.170620
Gen X         40258.576655
Gen Z         10294.943912
Millennials   26661.753886
Pre-Boomer    28800.826320
Name: adjusted_INCTOT, dtype: float64
```

```
In [22]: df.groupby('Generation_Category')['INCTOT'].median()
```

```
Out[22]: Generation_Category
Boomers      24000.0
Gen X         27000.0
Gen Z         9000.0
Millennials   20000.0
Pre-Boomer    14800.0
Name: INCTOT, dtype: float64
```

```
In [23]: import seaborn as sns
import matplotlib.pyplot as plt

#Relationships between housing and interest rate
fig, ax1 = plt.subplots()
```

```
line1 = sns.lineplot(x='YEAR', y='adjusted_MSPUS', data=df, ax=ax1, label='adjusted_MSPUS')
ax1.set_ylabel('adjusted_MSPUS')

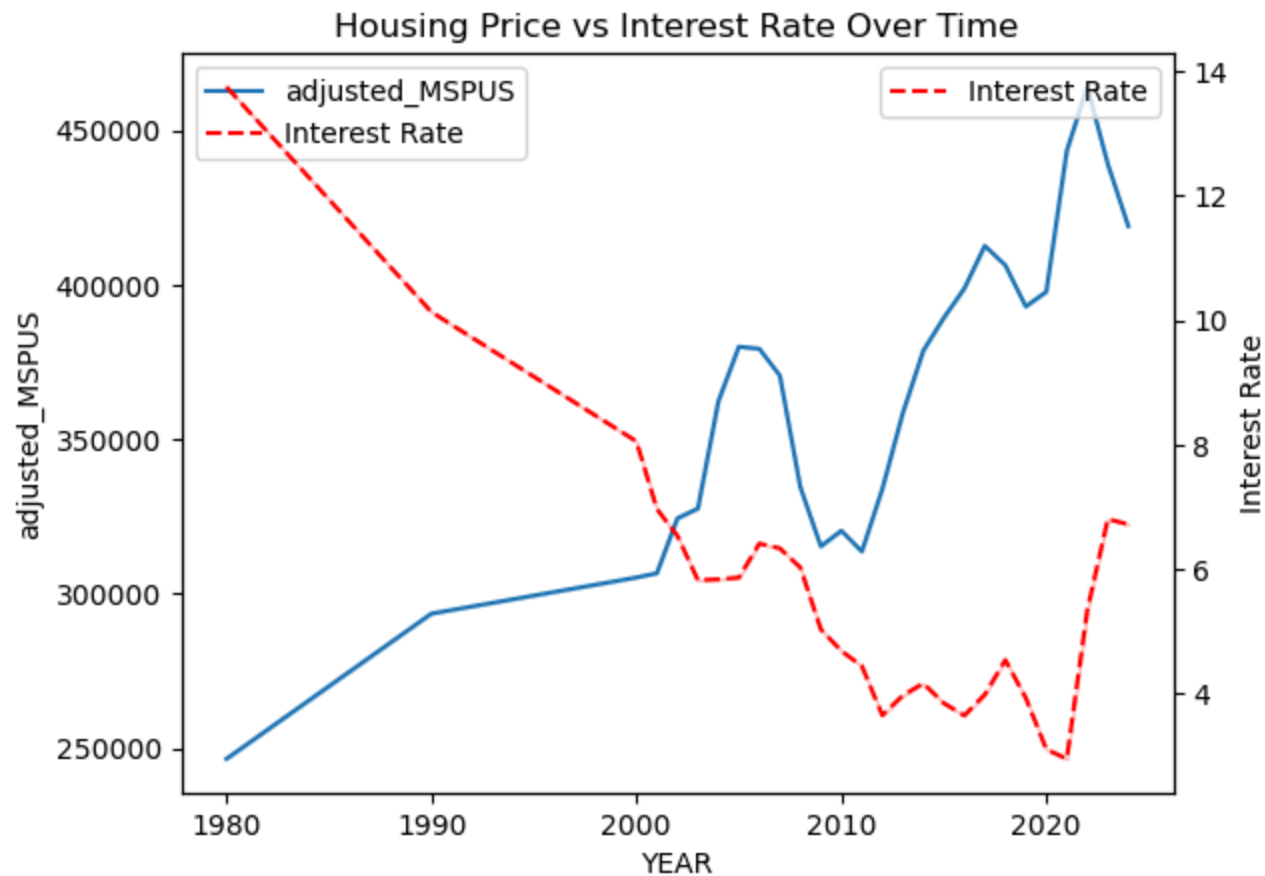
ax2 = ax1.twinx()

line2 = sns.lineplot(
    x='YEAR',
    y='fixed_rate_30',
    data=df,
    ax=ax2,
    linestyle='--',
    color='red',
    label='Interest Rate'
)
ax2.set_ylabel('Interest Rate')

lines = ax1.get_lines() + ax2.get_lines()
labels = [l.get_label() for l in lines]

ax1.legend(lines, labels, loc='upper left')

plt.title('Housing Price vs Interest Rate Over Time')
plt.show()
```

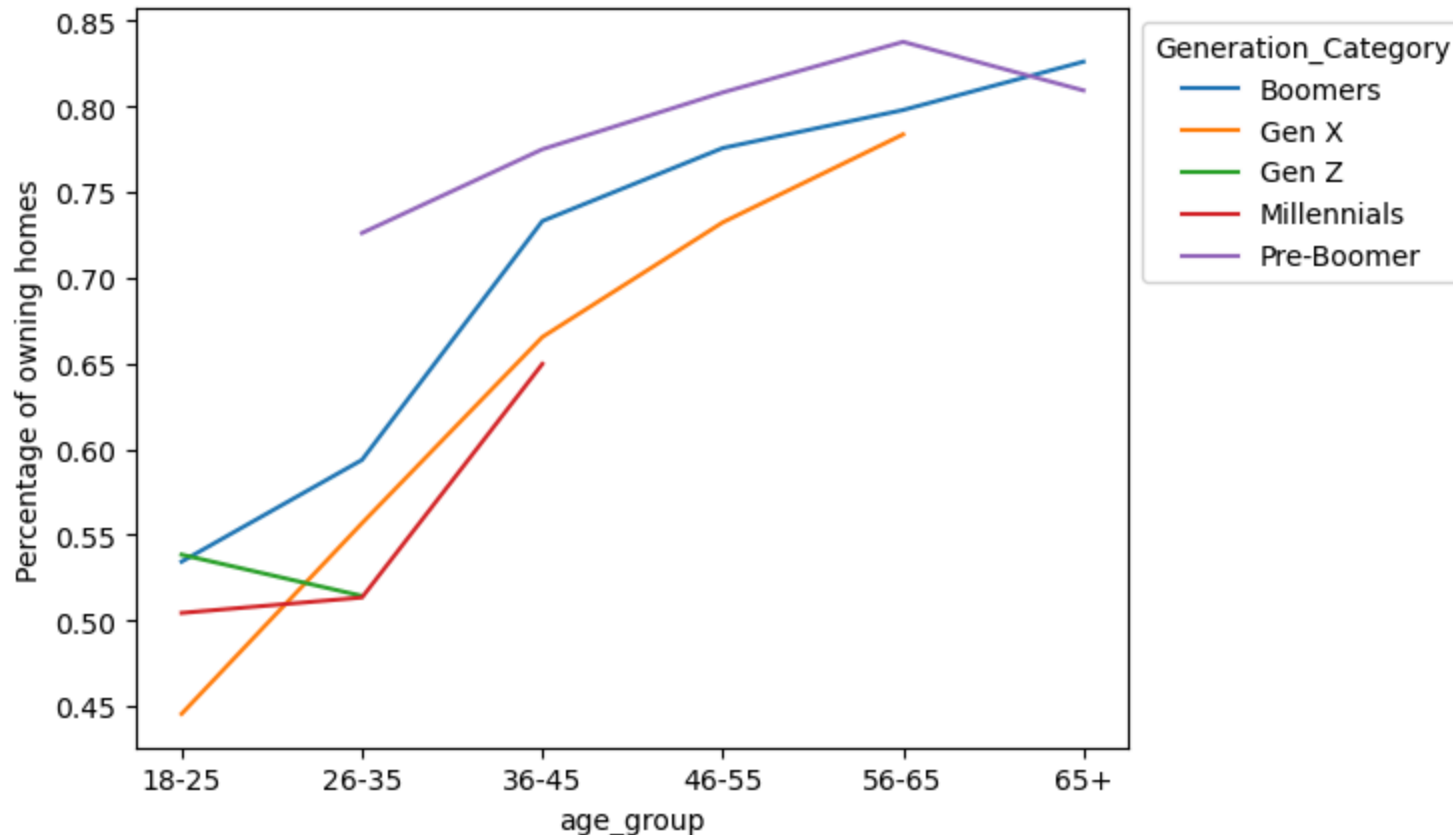


```
In [24]: #Percentage of owning homes by age groups and generations
diff = (df.groupby(['Generation_Category', 'age_group'], observed=True)
        .apply(lambda x: (x.loc[x['Ownership_Status']==1, 'PERWT'].sum() / x['PERWT'].sum(),
                          include_groups=False).reset_index(name='Percentage of owning homes'))
diff
```

Out[24]:

	Generation_Category	age_group	Percentage of owning homes
0	Boomers	18-25	0.534359
1	Boomers	26-35	0.593714
2	Boomers	36-45	0.733091
3	Boomers	46-55	0.775682
4	Boomers	56-65	0.797878
5	Boomers	65+	0.826084
6	Gen X	18-25	0.445548
7	Gen X	26-35	0.556802
8	Gen X	36-45	0.665412
9	Gen X	46-55	0.732306
10	Gen X	56-65	0.783636
11	Gen Z	18-25	0.538410
12	Gen Z	26-35	0.514367
13	Millennials	18-25	0.504417
14	Millennials	26-35	0.513435
15	Millennials	36-45	0.649726
16	Pre-Boomer	26-35	0.726106
17	Pre-Boomer	36-45	0.774966
18	Pre-Boomer	46-55	0.808135
19	Pre-Boomer	56-65	0.837648
20	Pre-Boomer	65+	0.809298

```
In [25]: #Percentage of owning homes by age groups and generations
sns.lineplot(x='age_group',y='Percentage of owning homes',hue='Generation_Category',data=diff)
plt.legend(bbox_to_anchor=(1,1),title='Generation_Category')
plt.show()
```



Interpretation

The plot shows that homeownership follows a consistent life-cycle pattern across all generations, rising steeply through the 30s and 40s before leveling off. However, Millennials and Gen Z own homes at notably lower rates at the same ages compared to Boomers and Gen X. This gap may be driven by structural factors such as student debt, delayed household formation, and housing prices rising faster than wages, which constrain ownership beyond what age and income alone would predict. Further analysis is required.

```
In [26]: df['age_centered'] = df['AGE'] - df['AGE'].mean()
df['age_c2'] = df['age_centered']**2
```

```
df["PERWT_norm"] = df["PERWT"] / df["PERWT"].sum() * len(df)
df = df[df['PERWT_norm']>0]
```

```
In [27]: df.head()
df = df.drop(['INCTOT', 'MSPUS', 'birth_year', 'cpi_factor'], axis=1)
```

```
In [28]: import statsmodels.api as sm
import statsmodels.formula.api as smf
import numpy as np

np.random.seed(123)
# split data into train/test
test_rows = np.random.choice(df.index, size=int(0.2 * len(df)), replace=False)

testData = df.loc[test_rows].reset_index(drop=True)
trainData = df.drop(test_rows).reset_index(drop=True)

#include all predictors
predictors = "+".join([
    col for col in trainData.columns
    if col not in ["Ownership_Status", "PERWT_norm"]
])
formula = f"Ownership_Status ~ {predictors}"

model_weight = smf.glm(
    formula=formula, data=trainData,
    family=sm.families.Binomial(),
    var_weights=trainData["PERWT_norm"]
).fit(cov_type='cluster', cov_kws={'groups': trainData['YEAR']})

print(model_weight.summary())
```

C:\Users\Administrator\anaconda3\Lib\site-packages\statsmodels\genmod\generalized_linear_model.py:1655: Specification Warning: cov_type not fully supported with var_weights
 warnings.warn('cov_type not fully supported with var_weights',

Generalized Linear Model Regression Results

=====						
Dep. Variable:	Ownership_Status	No. Observations:	616060			
Model:	GLM	Df Residuals:	616036			
Model Family:	Binomial	Df Model:	23			
Link Function:	Logit	Scale:	1.0000			
Method:	IRLS	Log-Likelihood:	-3.3866e+05			
Date:	Sun, 26 Apr 2026	Deviance:	6.7733e+05			
Time:	13:07:35	Pearson chi2:	6.30e+05			
No. Iterations:	6	Pseudo R-squ. (CS):	0.1345			
Covariance Type:	cluster					
=====						
	coef	std err	z	P> z	[0.025	0.975]

Intercept	0.0157	nan	nan	nan	nan	nan
Marital_Status[T.Never married]	-0.8481	0.015	-57.476	0.000	-0.877	-0.819
Marital_Status[T.Separated/Divorced]	-1.1134	0.032	-35.240	0.000	-1.175	-1.051
Marital_Status[T.Widowed]	-0.8228	0.053	-15.419	0.000	-0.927	-0.718
Degree[T.Degrees beyond a bachelors degree]	-0.0110	0.059	-0.186	0.852	-0.127	0.105
Degree[T.No college Degree]	-0.1954	0.033	-5.898	0.000	-0.260	-0.130
Employment_Status[T.Not in labor force]	0.0113	0.014	0.832	0.405	-0.015	0.038
Employment_Status[T.Unemployed]	-0.0894	0.030	-2.987	0.003	-0.148	-0.031
Generation_Category[T.Gen X]	-0.2196	0.028	-7.804	0.000	-0.275	-0.164
Generation_Category[T.Gen Z]	0.0813	0.126	0.643	0.520	-0.167	0.329
Generation_Category[T.Millennials]	-0.2097	0.068	-3.105	0.002	-0.342	-0.077
Generation_Category[T.Pre-Boomer]	0.1076	0.049	2.191	0.028	0.011	0.204
age_group[T.26-35]	-0.2478	0.034	-7.351	0.000	-0.314	-0.182
age_group[T.36-45]	0.2265	0.054	4.213	0.000	0.121	0.332
age_group[T.46-55]	0.5724	0.067	8.545	0.000	0.441	0.704
age_group[T.56-65]	0.8201	0.081	10.149	0.000	0.662	0.978
age_group[T.65+]	0.9706	0.092	10.515	0.000	0.790	1.151
YEAR	-0.0084	0.008	-1.020	0.308	-0.025	0.008
PERWT	-0.0014	0.000	-11.672	0.000	-0.002	-0.001
AGE	0.3750	nan	nan	nan	nan	nan
fixed_rate_30	-0.0276	0.021	-1.335	0.182	-0.068	0.013
adjusted_INCTOT	4.749e-06	2.18e-07	21.790	0.000	4.32e-06	5.18e-06
adjusted_MSPUS	7.664e-07	8.47e-07	0.904	0.366	-8.94e-07	2.43e-06
age_centered	-0.3791	nan	nan	nan	nan	nan
age_c2	-2.554e-05	4.56e-05	-0.560	0.576	-0.000	6.39e-05
=====						

Variable Selection

```
In [29]: import warnings
from statsmodels.tools.sm_exceptions import PerfectSeparationWarning

# YEAR was excluded as a predictor and used for clustering
# # PERWT is used as sampling weights, not as a predictor
variables = [
    'C(Marital_Status)', 'C(Degree)', 'C(Employment_Status)',
    'C(Generation_Category)', 'age_centered', 'age_c2',
    'adjusted_INCTOT', 'fixed_rate_30', 'adjusted_MSPUS'
]

selected = []
remaining = variables.copy()
aic_trace = []
current_aic = float('inf')
best_model = None

def fit_glm(predictors):
    formula = 'Ownership_Status ~ ' + ' + '.join(predictors)
    return smf.glm(
        formula=formula,
        data=trainData,
        family=sm.families.Binomial(),
        var_weights=trainData['PERWT_norm']
    ).fit()

while remaining:
    results = []

    for var in remaining:
        model = fit_glm(selected + [var])
        results.append((model.aic, var, model))

    best_aic, best_var, best_fit = min(results, key=lambda x: x[0])

    if best_aic >= current_aic:
```

break

```

selected.append(best_var)
remaining.remove(best_var)
current_aic = best_aic
best_model = best_fit
aic_trace.append((len(selected), best_aic, best_var))

print(f"Step {len(selected):2d}: + '{best_var:<35s}'  AIC = {best_aic:.2f}")

```

```

Step 1: + 'C(Marital_Status)          '  AIC = 717849.34
Step 2: + 'age_centered              '  AIC = 699756.82
Step 3: + 'adjusted_INCTOT          '  AIC = 692048.61
Step 4: + 'C(Generation_Category)    '  AIC = 689061.56
Step 5: + 'C(Degree)                 '  AIC = 688373.28
Step 6: + 'C(Employment_Status)      '  AIC = 688330.39
Step 7: + 'age_c2                    '  AIC = 688283.45

```

Interpretation

Only seven out of nine variables were retained by the forward selection procedure.

```

In [30]: model_weight_variable_section = smf.glm(
        'Ownership_Status ~ age_centered + age_c2 + adjusted_INCTOT + C(Marital_Status) + C(Degree) + C(Employment_Stat
        family=sm.families.Binomial(),
        var_weights=trainData["PERWT_norm"]
    ).fit(cov_type='cluster', cov_kws={'groups': trainData['YEAR']})

print(model_weight_variable_section.summary())

```

C:\Users\Administrator\anaconda3\Lib\site-packages\statsmodels\genmod\generalized_linear_model.py:1655: SpecificationWarning: cov_type not fully supported with var_weights
 warnings.warn('cov_type not fully supported with var_weights',

Generalized Linear Model Regression Results

=====						
Dep. Variable:	Ownership_Status	No. Observations:	616060			
Model:	GLM	Df Residuals:	616045			
Model Family:	Binomial	Df Model:	14			
Link Function:	Logit	Scale:	1.0000			
Method:	IRLS	Log-Likelihood:	-3.4413e+05			
Date:	Sun, 26 Apr 2026	Deviance:	6.8825e+05			
Time:	13:12:11	Pearson chi2:	6.24e+05			
No. Iterations:	6	Pseudo R-squ. (CS):	0.1190			
Covariance Type:	cluster					
=====						
	coef	std err	z	P> z	[0.025	0.975]

Intercept	1.4399	0.036	39.995	0.000	1.369	1.510
C(Marital_Status)[T.Never married]	-0.8127	0.021	-37.976	0.000	-0.855	-0.771
C(Marital_Status)[T.Separated/Divorced]	-1.1164	0.016	-67.966	0.000	-1.149	-1.084
C(Marital_Status)[T.Widowed]	-0.8911	0.023	-38.907	0.000	-0.936	-0.846
C(Degree)[T.Degrees beyond a bachelors degree]	-0.0003	0.018	-0.018	0.986	-0.036	0.036
C(Degree)[T.No college Degree]	-0.1895	0.017	-11.338	0.000	-0.222	-0.157
C(Employment_Status)[T.Not in labor force]	0.0241	0.012	1.943	0.052	-0.000	0.048
C(Employment_Status)[T.Unemployed]	-0.0826	0.030	-2.761	0.006	-0.141	-0.024
C(Generation_Category)[T.Gen X]	-0.4088	0.050	-8.145	0.000	-0.507	-0.310
C(Generation_Category)[T.Gen Z]	0.0891	0.082	1.093	0.274	-0.071	0.249
C(Generation_Category)[T.Millennials]	-0.3854	0.065	-5.967	0.000	-0.512	-0.259
C(Generation_Category)[T.Pre-Boomer]	0.0767	0.038	2.032	0.042	0.003	0.151
age_centered	0.0183	0.001	12.993	0.000	0.016	0.021
age_c2	-8.515e-05	4.1e-05	-2.077	0.038	-0.000	-4.78e-06
adjusted_INCTOT	4.614e-06	2.07e-07	22.304	0.000	4.21e-06	5.02e-06
=====						

In [31]: `from scipy.stats import chi2`

```
# Full model: includes Generation_Category
full_model = smf.glm(
    'Ownership_Status ~ age_centered + age_c2 + adjusted_INCTOT '
    '+ C(Marital_Status) + C(Degree) + C(Employment_Status) '
    '+ C(Generation_Category)',
    data=trainData,
    family=sm.families.Binomial(),
    var_weights=trainData['PERWT_norm']
).fit()
```

```
# Reduced model: without Generation_Category
reduced_model = smf.glm(
    'Ownership_Status ~ age_centered + age_c2 + adjusted_INCTOT '
    '+ C(Marital_Status) + C(Degree) + C(Employment_Status)',
    data=trainData,
    family=sm.families.Binomial(),
    var_weights=trainData['PERWT_norm']
).fit()

lr_stat = reduced_model.deviance - full_model.deviance
df_c = reduced_model.df_resid - full_model.df_resid
p_value = 1 - chi2.cdf(lr_stat, df_c)

print("Generation LRT")
print("LR statistic:", lr_stat)
print("df:", df_c)
print("p-value:", p_value)
```

Generation LRT

LR statistic: 3239.1517844548216

df: 4

p-value: 0.0

Interpretation

The p-value is 0.0, which is less than 0.05. Therefore, we reject the null hypothesis and conclude that Generation_Category is statistically significant predictor of the homeowner_status. This indicates that generation category significantly improves model fit after controlling for age, income, marital status, education, and employment status.

In [32]: *#Checking if full model fits better than an intercept only model*

```
full_model = model_weight_variable_section

null_model = smf.glm(
    'Ownership_Status ~ 1',
    data=trainData,
    family=sm.families.Binomial(),
    var_weights=trainData['PERWT_norm']
).fit()
```

```
LR_stat = 2 * (full_model.llf - null_model.llf)
df_diff = full_model.df_model - null_model.df_model
p_value = chi2.sf(LR_stat, df_diff)

print("LR statistic:", LR_stat)
print("Degrees of freedom:", df_diff)
print("p-value:", p_value)
```

LR statistic: 78057.98492428893

Degrees of freedom: 14

p-value: 0.0

Interpretation

Since the p-value is less than 0.05, we reject the null hypothesis that all coefficients are equal to zero, and conclude that the model is statistically significant overall and provides a significantly better fit than the model with no predictors.

Statistical analysis and hypothesis testing

Marital Status

If the Marital_Status is Never married, the log odds of owning a home decreases by 0.8127 as compared to when the individual is married and the odds of owning a home change by a factor of $\exp(-0.8127) = 0.44$ as compared to when the individual is married, holding all other variables constant.

If the Marital_Status is Separated/Divorced, the log odds of owning a home decreases by 1.1164 as compared to when the individual is married and the odds of owning a home change by a factor of $\exp(-1.1164) = 0.33$ as compared to when the individual is married, holding all other variables constant.

If the Marital_Status is Widowed, the log odds of owning a home decreases by 0.8911 as compared to when the individual is married and the odds of owning a home change by a factor of $\exp(-0.8911) = 0.41$ as compared to when the individual is married, holding all other variables constant.

Null hypothesis: $H_0: \text{Marital Status} = 0$

Alternative hypothesis: $H_A: \text{Marital_Status} \neq 0$

Since the p-values for all Marital_Status categories are less than 0.05, we reject the null hypothesis that marital status has no effect on homeownership. This indicates that marital status has a statistically significant effect on homeownership.

Income

For a one-unit increase in adjusted_INCTOT, the log odds of owning a home increases by $4.614e-06$ and the odds of owning a home increase by $\exp(4.614e-06) \approx 1.000006$, holding all other variables constant.

Null hypothesis: $H_0: \text{Income} = 0$

Alternative hypothesis: $H_A: \text{Income} \neq 0$

Since the p-value for INCTOT is less than 0.05, we reject the null hypothesis that income has no effect on homeownership. This suggests that income is a statistically significant predictor of homeownership.

Education

If the Degree is No college Degree, the log odds of owning a home decreases by 0.1895 as compared to when the individual has a bachelor's degree (baseline) and the odds of owning a home change by a factor of $\exp(-0.1895) = 0.83$ as compared to the baseline, holding all other variables constant.

If the Degree is Degrees beyond a bachelor's degree, the log odds of owning a home decreases by 0.0003 as compared to the baseline and the odds of owning a home change by a factor of $\exp(-0.0003) \approx 1.00$ as compared to the baseline, holding all other variables constant.

Null hypothesis: $H_0: \text{Education} = 0$

Alternative hypothesis: $H_A: \text{Education} \neq 0$

Since the p-value for individuals with no college degree is less than 0.05, we reject the null hypothesis that education has no effect on homeownership. This indicates that lacking a college degree significantly affects homeownership.

Employment Status

If the Employment_Status is Unemployed, the log odds of owning a home decreases by 0.0826 as compared to when the individual is employed and the odds of owning a home change by a factor of $\exp(-0.0826) = 0.92$ as compared to when the individual is employed, holding all other variables constant.

If the Employment_Status is Not in labor force, the log odds of owning a home increases by 0.0241 as compared to when the individual is employed and the odds of owning a home change by a factor of $\exp(0.0241) = 1.02$ as compared to when the individual is employed, holding all other variables constant.

Null hypothesis: H_0 : Employment Status=0

Alternative hypothesis: H_A : Employment Status!=0

Since the p-value for the Unemployed category is less than 0.05, we reject the null hypothesis that employment status has no effect on homeownership. This indicates that being unemployed significantly affects homeownership likelihood.

Generation Category

If the Generation_Category is Gen X, the log odds of owning a home decreases by 0.4088 as compared to the baseline generation_Boomer and the odds of owning a home change by a factor of $\exp(-0.4088) = 0.66$ as compared to the baseline, holding all other variables constant.

If the Generation_Category is Millennials, the log odds of owning a home decreases by 0.3854 as compared to the baseline generation_Boomer and the odds of owning a home change by a factor of $\exp(-0.3854) = 0.68$ as compared to the baseline, holding all other variables constant.

If the Generation_Category is Gen Z, the log odds of owning a home increases by 0.0891 as compared to the baseline generation_Boomer and the odds of owning a home change by a factor of $\exp(0.0891) = 1.09$ as compared to the baseline, holding all other variables constant. //The positive Gen Z coefficient should be interpreted carefully. Gen Z's presence in the sample is limited to early adulthood, and the observed effect likely reflects selection bias.//

If the Generation_Category is Pre-Boomer, the log odds of owning a home increases by 0.0767 as compared to the baseline generation_Boomer and the odds of owning a home change by a factor of $\exp(0.0767) = 1.08$ as compared to the baseline, holding all other variables constant.

Null hypothesis: H_0 : Generation Category=0

Alternative hypothesis: H_A : Generation Category!=0

Since the p-values for Gen X and Millennials are less than 0.05, we reject the null hypothesis that these generational categories have no effect on homeownership. This suggests that these generations differ significantly from the baseline group.

Age (Nonlinear Effect)

For a one-unit increase in age_centered, the log odds of owning a home increases by 0.0183 and the odds of owning a home increase by $\exp(0.0183) = 1.02$, holding all other variables constant.

For a one-unit increase in age_c2, the log odds of owning a home decreases by $8.515e-05$ and the odds of owning a home change by a factor of $\exp(-8.515e-05) \approx 0.9999$, holding all other variables constant.

Null hypothesis: H_0 : Age=0

Alternative hypothesis: H_A : Age!=0

Since the p-values for both age_centered and age_c2 are less than 0.05, we reject the null hypothesis that age has no effect on homeownership. This indicates that age has a statistically significant nonlinear effect on homeownership.

Goodness of Fit testing and model assumption checks // Outlier screening and handling procedures

```
In [33]: from scipy.stats import chi2

deviance = model_weight_variable_section.deviance
df_resid = model_weight_variable_section.df_resid

p_value = chi2.sf(deviance, df_resid)

print("Deviance:", deviance)
print("DF Residual:", df_resid)
print("GOF p-value:", p_value)
```

Deviance: 688253.4526076193
DF Residual: 616045
GOF p-value: 0.0

```
In [34]: pearson_chi2 = model_weight_variable_section.pearson_chi2  
df_resid = model_weight_variable_section.df_resid  
  
p_value = chi2.sf(pearson_chi2, df_resid)  
  
print("Pearson Chi2:", pearson_chi2)  
print("p-value:", p_value)
```

Pearson Chi2: 623721.4552349063
p-value: 2.8323917021116067e-12

Interpretation Null Hypothesis: H0: The model is a good fit

Alternative Hypothesis: HA: The model is not a good fit

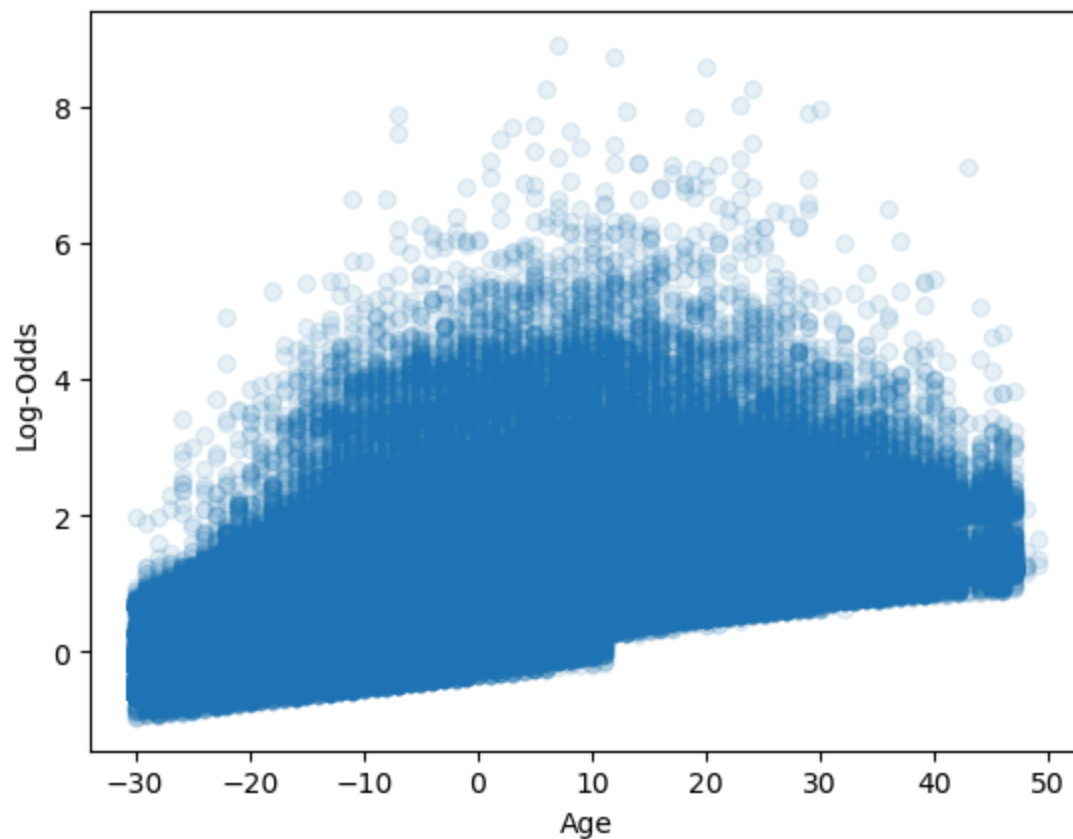
A goodness-of-fit test was conducted using both deviance and Pearson chi-square tests. Since both p values are less than 0.05, we reject the null hypothesis and conclude that the model is not a good fit.

```
In [35]: print(f"Dispersion estimate (Pearson chi2 / df): {pearson_chi2 / df_resid:.4f}")
```

Dispersion estimate (Pearson chi2 / df): 1.0125

Interpretation Since the dispersion estimate is close to 1, there is no evidence of overdispersion, indicating that the variance is appropriately specified.

```
In [36]: p = model_weight_variable_section.predict(trainData)  
logit_p = np.log(p / (1 - p))  
  
plt.scatter(trainData['age_centered'], logit_p, alpha=0.1)  
plt.xlabel("Age")  
plt.ylabel("Log-Odds")  
plt.show()
```



Interpretation

The relationship between age and the log-odds of homeownership appears approximately linear, although some dispersion is observed. This suggests that the linearity assumption is reasonably satisfied, with minor deviations likely due to interactions with other predictors.

```
In [37]: from statsmodels.stats.outliers_influence import variance_inflation_factor
X = model_weight_variable_section.model.exog
names = model_weight_variable_section.model.exog_names

vif_df=pd.DataFrame()
vif_df['variable']=names
vif_df['VIF']=[variance_inflation_factor(X,i) for i in range(X.shape[1])]
vif_df
```

Out[37]:

	variable	VIF
0	Intercept	11.130489
1	C(Marital_Status)[T.Never married]	1.551483
2	C(Marital_Status)[T.Separated/Divorced]	1.074426
3	C(Marital_Status)[T.Widowed]	1.303080
4	C(Degree)[T.Degrees beyond a bachelors degree]	1.460921
5	C(Degree)[T.No college Degree]	1.539933
6	C(Employment_Status)[T.Not in labor force]	1.549263
7	C(Employment_Status)[T.Unemployed]	1.045127
8	C(Generation_Category)[T.Gen X]	1.438707
9	C(Generation_Category)[T.Gen Z]	1.281597
10	C(Generation_Category)[T.Millennials]	1.775517
11	C(Generation_Category)[T.Pre-Boomer]	1.832796
12	age_centered	3.017904
13	age_c2	1.634386
14	adjusted_INCTOT	1.287977

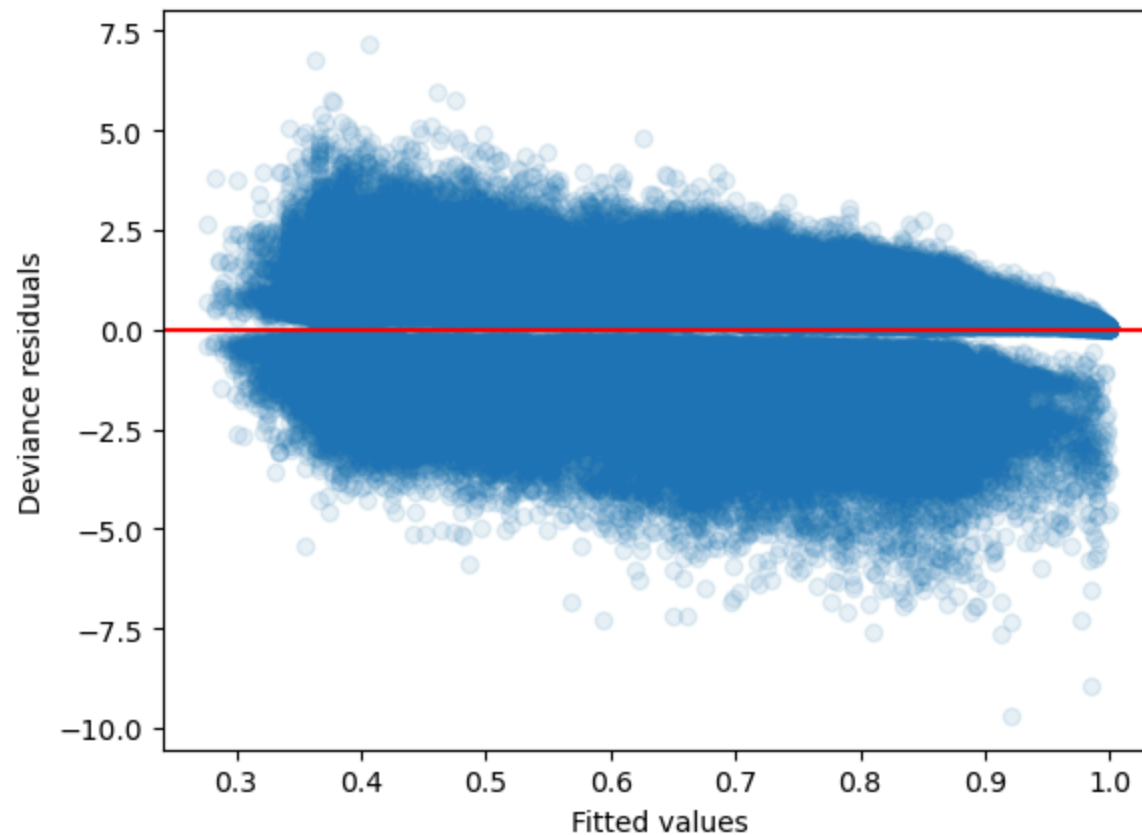
Interpretation

Variance Inflation Factors (VIFs) were examined and found to be within acceptable ranges(<10), indicating no severe multicollinearity among predictors.

```
In [38]: resid = model_weight_variable_section.resid_deviance

plt.scatter(model_weight_variable_section.fittedvalues, resid, alpha=0.1)
plt.axhline(0, color='red')
plt.xlabel("Fitted values")
```

```
plt.ylabel("Deviance residuals")  
plt.show()
```



Interpretation

The residual plot shows that deviance residuals are generally centered around zero, with no strong nonlinear patterns, suggesting that the model is reasonably well specified. However, a slight downward trend is observed as fitted values increase, indicating mild systematic bias where the model may overpredict probabilities at higher fitted values.

```
In [39]: influence = model_weight_variable_section.get_influence(observed=False)  
         cooks_d = influence.cooks_distance[0]  
  
         threshold = 4 / len(trainData)  
         outliers = cooks_d > threshold
```

```
top_5 = cooks_d.argsort()[-5:][::-1]

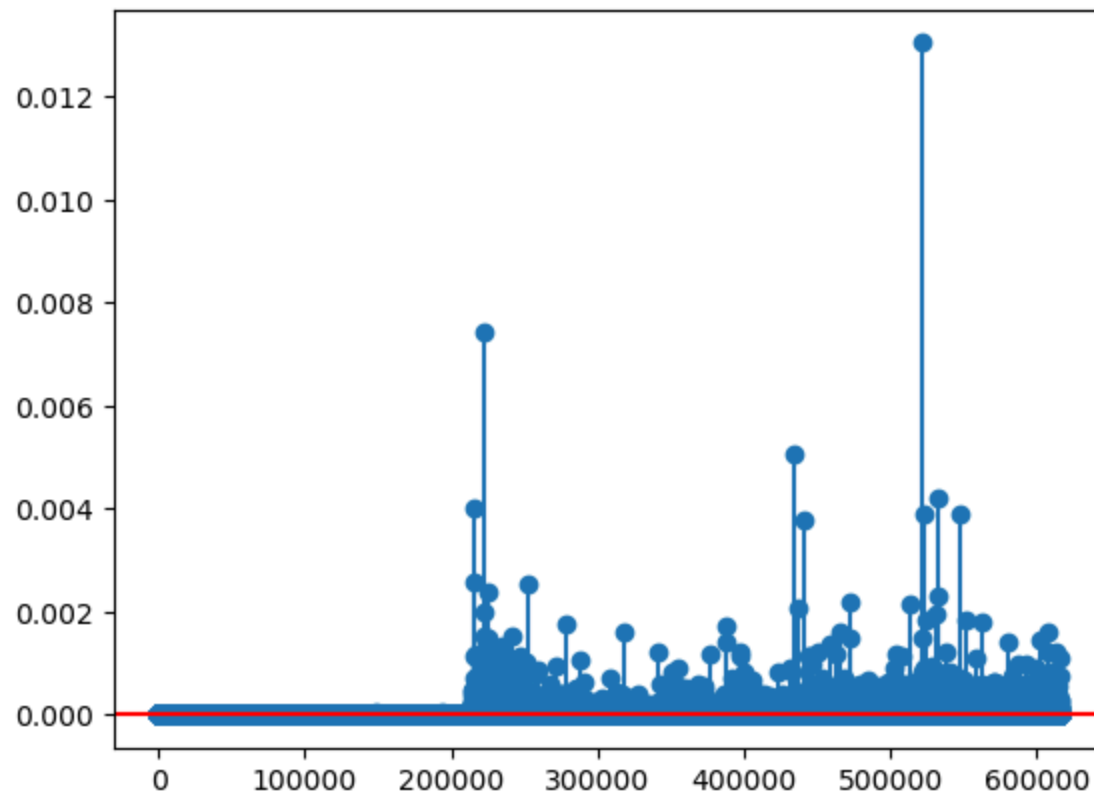
out = trainData.iloc[top_5].copy()
out['cooks_d'] = cooks_d[top_5]

print("Cook's D threshold:", threshold)
print("Number of flagged observations:", outliers.sum())
out

plt.stem(cooks_d)
plt.axhline(threshold, color='red')
plt.show()
```

Cook's D threshold: 6.492874070707399e-06

Number of flagged observations: 63906



```
In [40]: #top 5 outliers
top_5=cooks_d.argsort()[-5:][::-1]
```

```
out=trainData.iloc[top_5]
out
```

Out[40]:

	YEAR	PERWT	AGE	fixed_rate_30	Ownership_Status	Marital_Status	Degree	Employment_Status	Generation_C
521827	2020	1476.0	70	3.111698	0	Separated/Divorced	Bachelors Degree	Employed	I
222353	2002	771.0	59	6.537308	0	Married	Degrees beyond a bachelors degree	Employed	Pre-
434536	2015	400.0	56	3.850577	0	Separated/Divorced	Bachelors Degree	Employed	I
532318	2020	1556.0	77	3.111698	0	Separated/Divorced	No college Degree	Unemployed	Pre-
215368	2001	2206.0	41	6.967885	0	Never married	No college Degree	Unemployed	I

Interpretation

These observations fall within plausible real world ranges and do not appear to be data entry errors. Moreover, the sample size is large and the influential observations do not change the model interpretation, I decided to keep them in the model.

```
In [41]: train_prob = model_weight_variable_section.predict(trainData)
test_prob = model_weight_variable_section.predict(testData)

# I used a threshold of 0.6 to improve classification of non-homeowners,
# since the default 0.5 threshold outputed weaker recall for the non-homeowner class.
train_pred = (train_prob >= 0.6).astype(int)
test_pred = (test_prob >= 0.6).astype(int)
```

```
In [42]: from sklearn.metrics import (
    accuracy_score,
    precision_score,
```

```
recall_score,  
f1_score,  
roc_auc_score,  
confusion_matrix,  
classification_report  
  
)  
  
def evaluate_model(y_true, y_prob, y_pred, dataset_name="Dataset"):  
    print(f"\n{' '*30}")  
    print(f"{dataset_name} Performance")  
    print(f"{' '*30}")  
  
    print("Accuracy :", round(accuracy_score(y_true, y_pred), 5))  
    print("Precision:", round(precision_score(y_true, y_pred, zero_division=0), 5))  
    print("Recall   :", round(recall_score(y_true, y_pred, zero_division=0), 5))  
    print("F1 Score :", round(f1_score(y_true, y_pred, zero_division=0), 5))  
    print("ROC AUC  :", round(roc_auc_score(y_true, y_prob), 5))  
    print("\nConfusion Matrix:")  
    print(confusion_matrix(y_true, y_pred))  
  
    print("\nClassification Report:")  
    print(classification_report(y_true, y_pred, zero_division=0))  
  
evaluate_model(trainData['Ownership_Status'], train_prob, train_pred, "Train")  
evaluate_model(testData['Ownership_Status'], test_prob, test_pred, "Test")
```

=====

Train Performance

=====

Accuracy : 0.71778

Precision: 0.79777

Recall : 0.82457

F1 Score : 0.81095

ROC AUC : 0.71852

Confusion Matrix:

[[69285 94530]

[79336 372909]]

Classification Report:

	precision	recall	f1-score	support
0	0.47	0.42	0.44	163815
1	0.80	0.82	0.81	452245
accuracy			0.72	616060
macro avg	0.63	0.62	0.63	616060
weighted avg	0.71	0.72	0.71	616060

=====

Test Performance

=====

Accuracy : 0.71766

Precision: 0.79883

Recall : 0.82362

F1 Score : 0.81104

ROC AUC : 0.7174

Confusion Matrix:

[[17213 23500]

[19984 93317]]

Classification Report:

	precision	recall	f1-score	support
0	0.46	0.42	0.44	40713
1	0.80	0.82	0.81	113301

accuracy			0.72	154014
macro avg	0.63	0.62	0.63	154014
weighted avg	0.71	0.72	0.71	154014

Interpretation

The model shows moderate predictive performance (ROC AUC = 0.71) and generalizes well on the test set, as indicated by nearly identical performance across training and testing data. It achieves solid performance in identifying homeowners, with high precision (=0.80) and recall (=0.82), resulting in a strong F1 score. For non-homeowners, the model shows lower recall (=0.42), indicating that a portion of this group is still misclassified. Overall, the model captures meaningful patterns in the data, but its predictive strength remains stronger for the majority class(1) than for the minority class(0).

```
In [43]: import warnings
import statsmodels.api as sm
import statsmodels.formula.api as smf
from sklearn.model_selection import StratifiedKFold
from sklearn.metrics import roc_auc_score, log_loss, brier_score_loss
from scipy.stats import chi2
warnings.filterwarnings('ignore')

FORMULA = (
    'Ownership_Status ~ age_centered + age_c2 + adjusted_INCTOT '
    '+ C(Marital_Status) + C(Degree) + C(Employment_Status) + C(Generation_Category)'
)

N_FOLDS = 5
skf = StratifiedKFold(n_splits=N_FOLDS, shuffle=True, random_state=123)
y = df['Ownership_Status'].values

aucs = []

for fold, (train_idx, test_idx) in enumerate(skf.split(df, y)):
    train_df = df.iloc[train_idx]
    test_df = df.iloc[test_idx]

    model_cv = smf.glm(
        FORMULA,
        data=train_df,
```

```
family=sm.families.Binomial(),
var_weights=train_df['PERWT_norm']
).fit()

y_prob = model_cv.predict(test_df)
y_true = test_df['Ownership_Status'].values

aucs.append(roc_auc_score(y_true, y_prob))

print(f"Fold {fold+1}: AUC={aucs[-1]:.4f} ")

print(f"\n--- 5-Fold CV Summary ---")
print(f"AUC:      mean={np.mean(aucs):.4f}  std={np.std(aucs):.4f}")
```

Fold 1: AUC=0.7173

Fold 2: AUC=0.7174

Fold 3: AUC=0.7191

Fold 4: AUC=0.7185

Fold 5: AUC=0.7190

--- 5-Fold CV Summary ---

AUC: mean=0.7183 std=0.0008